An Empirical Evaluation of AI-Powered Coding Agents: A Case Study of Claude Code with deepseek-v4-flash on Software Development Tasks

Sun Chengxi
Independent Researcher
3852236135@qq.com

Abstract—AI-powered coding agents are increasingly used for software development tasks, yet their behavior across different task types remains insufficiently characterized. This paper presents a small-scale empirical case study of an agentic coding setup using Claude Code as the orchestration environment with a DeepSeek-based backend. We evaluate the setup on 18 software development tasks covering algorithm implementation, bug fixing, refactoring, and test writing across JavaScript, TypeScript, Go, and JSX. In the primary single-run evaluation, within this small-scale benchmark, the agent completed all 18 tasks and passed all 162 predefined test assertions. To assess run-to-run stability, we further repeated six representative tasks five times each. Five tasks achieved stable success across all repeated runs, whereas a distributed-lock bug-fixing task succeeded in only 3 out of 5 runs, revealing substantial variability for complex concurrent logic. These findings suggest that agentic coding systems can perform strongly on well-specified, common programming tasks, but single-run evaluation may overestimate reliability for concurrency-heavy or distributed-system tasks. We discuss threats to validity including benchmark size, training-data contamination, absence of human baselines, and limitations of test-based evaluation.

Index Terms—AI coding agents, large language models, empirical software engineering, automated programming, Claude Code, deepseek-v4-flash

I. Introduction

A. Background and Motivation

Large language models (LLMs) capable of generating, understanding, and manipulating source code have fundamentally changed software development practices. From early code completion tools [1] to contemporary AI-powered coding agents, the trajectory of automated programming has accelerated dramatically in recent years. Modern AI coding agents represent a significant evolution beyond simple code completion: they can interpret natural language requirements, generate complete implementations, diagnose and fix bugs, restructure existing codebases, and write comprehensive test suites autonomously [2], [3], [11].

The potential impact of these tools on software development productivity is substantial. Studies have shown that AI-assisted programming can reduce development time by 35–55% for certain tasks [4], [5], and that professional developers increasingly rely on these tools for routine coding activities [6]. However, the empirical evaluation of AI coding agents faces several methodological challenges. Most existing evaluations focus on isolated code generation benchmarks [7], [8] or specific tasks within narrow domains [9], [11], leaving questions about generalizability across diverse software engineering activities.

Furthermore, the rapid evolution of underlying models, from GPT-based architectures [10] to specialized code models like Codex [2] and open-source alternatives [3], [11], necessitates continuous empirical reassessment. The integration of tool-use abilities and multi-turn interaction patterns in modern coding agents demands evaluation frameworks that capture both the breadth and depth of real-world software development.

B. Research Questions and Contributions

This study addresses the following research questions:

RQ1: Effectiveness Across Task Types: How effectively does an AI-powered coding agent perform across diverse software development task categories, including algorithm implementation, bug fixing, refactoring, and test writing?

RQ2: Difficulty Scaling: How does agent performance vary with task difficulty across simple, medium, and hard levels?

RQ3: Language Versatility: How does the agent handle multiple programming languages with varying paradigms and concurrency models?

RQ4: Quality and Correctness: What is the quality of generated code in terms of test pass rates, edge-case handling, and adherence to task specifications?

The primary contributions of this paper are:

1) A systematically designed benchmark of 18 software development tasks spanning four task types and three difficulty levels, implementing a balanced evaluation framework for AI coding agents.
2) A comprehensive empirical evaluation of Claude Code with deepseek-v4-flash, achieving 100% task pass rate and 100% test assertion pass rate across 162 tests in four programming languages in single-run evaluation, complemented by a variability analysis across 30 repeated runs.
3) A detailed analysis of the agent's performance characteristics across different task categories, difficulty lev-

els, and programming languages, identifying strengths and patterns in automated code generation.

4) A discussion of methodological considerations for evaluating AI coding agents, including limitations of single-run evaluations and the need for human baselines and statistical rigor.

C. Paper Organization

The remainder of this paper is organized as follows. Section II describes the experimental methodology, including task design, evaluation metrics, and experimental setup. Section III presents the experimental results and statistical analysis. Section IV discusses the findings, implications, and limitations. Section V reviews related work in AI-assisted programming and empirical software engineering. Section VI concludes the paper and outlines directions for future research.

II. Methodology

A. Task Design

We designed a benchmark suite of 18 software development tasks organized along two dimensions: task type and difficulty level. The four task types represent core software engineering activities:

- Algorithm Implementation (5 tasks): Implementing standard algorithms and data structures from natural language specifications.
- Bug Fixing (5 tasks): Diagnosing and repairing intentionally introduced bugs in existing code.
- Code Refactoring (4 tasks): Restructuring existing code to improve design, performance, or maintainability without changing external behavior.
- Test Writing (4 tasks): Creating comprehensive test suites for existing code components.

Each task type includes tasks at three difficulty levels—simple (S), medium (M), and hard (H)—operationalized through specific complexity criteria. Table I presents the complete task taxonomy.

TABLE I: Task Taxonomy and Descriptions

ID	Type	Description	Lang.
S1	Algorithm	Two-Sum problem using hash map O(n)	JS
S2	Algorithm	Valid parentheses using stack O(n)	JS
S3	Bug Fix	Fix infinite loop and integer overflow in binary search	JS
S4	Bug Fix	Fix React closure trap with useRef	JSX
S5	Refactor	Decompose spaghetti code into 7 functions	JS
S6	Testing	Unit tests for 3 utility functions (26 cases)	JS
M1	Algorithm	LRU cache with doubly linked list + hash map O(1)	JS
M2	Algorithm	Topological sort using three-color DFS	JS
M3	Bug Fix	Fix concurrent map read/write panic with sync.RWMutex	Go
M4	Bug Fix	Fix SQL injection with parameterized queries + bcrypt	JS
M5	Refactor	Refactor conditionals into 6 strategy classes	JS
M6	Refactor	Convert synchronous I/O to async with fs.promises	JS
M7	Testing	Integration tests for REST API with supertest (18 cases)	JS
M8	Testing	Unit tests for database operations with Jest mocks (13 cases)	TS
H1	Algorithm	Red-black tree insertion with rotation and recoloring	JS
H2	Bug Fix	Fix distributed lock deadlock with Lua script + watchdog	Go
H3	Refactor	Refactor monolith into Saga orchestrator with 5 microservices	TS
H4	Testing	Concurrency stress tests with race detection	Go

B. Difficulty Classification

Task difficulty was determined by a combination of factors:

Simple tasks involved single-algorithm implementations requiring standard library usage, bugs with straightforward root causes (e.g., off-by-one errors, missing boundary checks), isolated refactoring of single functions (5–15 lines), or basic unit tests covering 1–3 functions. These tasks typically require 10–30 lines of code and test 4–10 assertions per task.

Medium tasks required compound data structures (e.g., LRU cache combining hash map and linked list), concurrency bugs requiring synchronization primitives, architectural refactoring involving design patterns (e.g., Strategy pattern), or integration-level testing (e.g., HTTP API testing). These tasks typically require 30–80 lines of code and involve 3–18 assertions per task.

Hard tasks involved complex self-balancing tree algorithms (red-black tree with 6+ rotation cases), distributed system bugs requiring multi-component fixes (e.g., distributed lock with Lua scripting and watchdog goroutines), architectural decomposition into microservice patterns with Saga transaction orchestration, or concurrent stress testing with race detection. These tasks typically require 80–200+ lines of code and test 6–8 assertions per task.

C. Evaluation Metrics

We employed a multi-dimensional evaluation framework with both objective and analytical metrics:

1) Primary Metrics:

- Task Pass Rate: Binary indicator of whether the generated solution passes all acceptance criteria for the task.
- Test Assertion Pass Rate: The proportion of individual test assertions that pass, providing fine-grained correctness assessment.

2) Secondary Analytical Dimensions:

- Type Coverage: Analysis of success rates across the four task categories.

- Difficulty Scaling: Performance trajectory across simple, medium, and hard tasks.
- Language Robustness: Consistency of code generation quality across JavaScript, TypeScript, Go, and JSX.
- Edge-Case Sensitivity: Handling of boundary conditions, error states, and exceptional inputs.

D. Experimental Setup

The experiment was conducted using Claude Code framework, an AI-powered coding orchestration environment developed by Anthropic. The underlying language model used was deepseek-v4-flash, accessed through an API proxy configuration that routes Claude Code's model requests to the DeepSeek API endpoint. This configuration is achieved by setting the ANTHROPIC_BASE_URL environment variable to point to a DeepSeek-compatible API endpoint, effectively substituting the default Claude model with deepseek-v4-flash while preserving Claude Code's agentic framework capabilities—including file system operations, code execution, and multi-turn interaction management. This setup enables empirical evaluation of the Claude Code frameworkic framework with different underlying model architectures.

1) Model Configuration: The deepseek-v4-flash model was accessed through the API proxy without explicit parameter tuning; default settings were used for both the agentic framework and the underlying model. The estimated inference parameters for the deepseek-v4-flash API endpoint were: temperature of approximately 0.7 (default value; not explicitly overridden), top-p of 1.0, and maximum output tokens determined by the model's context window configuration. These parameters are reported as estimated defaults because the API proxy layer does not expose the exact parameter values forwarded to the model endpoint. The Claude Code version was determined at runtime via claude –version during each experiment execution.

2) Interaction Protocol: Multi-turn interaction was inherently enabled by the Claude Code frameworkic framework. The agent was provided with the task specification as a single input file (Markdown format) and was permitted to engage in iterative code generation, testing, and refinement through its built-in tool-use capabilities—including file editing, command execution, and output analysis. Each task was executed in an isolated session with no access to previous runs or pre-existing solutions. The general prompt template used for all tasks was the task's Markdown specification file (see Table I for descriptions), which included the natural language problem description, language requirements, and acceptance criteria. No chain-of-thought prompting, few-shot examples, or system-level instructions were provided beyond the task description itself.

3) Evaluation Protocol: The evaluation was performed as follows:

- Execution Environment: The coding agent was provided with each task's natural language specification as input, without example solutions or hints beyond the task description. No few-shot examples were provided.
- Task Execution: For each task, the agent produced a solution in a single interaction session. The agent was permitted to write, test, and iterate on code within the session. The generated code was then evaluated against a predefined test suite, with each test assertion independently verified.
- Hardware and Software: Experiments were conducted on a standard development workstation running Ubuntu 22.04 with an Intel i7 processor and 32 GB RAM. The Claude Code framework operated with default settings (no custom configuration files or environment overrides beyond the API proxy URL).
- Data Collection: For each task, we recorded the task identifier, type, complexity level, pass/fail status, number of tests passed, total tests, programming language, duration in seconds, token consumption, number of interaction turns, and qualitative notes on the solution approach.

III. Results

A. Overall Performance

Within our small-scale benchmark, the Claude Code framework achieved a 100% task pass rate, successfully completing all 18 tasks across the four categories and three difficulty levels. A total of 162 individual test assertions were executed, all of which passed (100%). Table II summarizes the aggregate results.

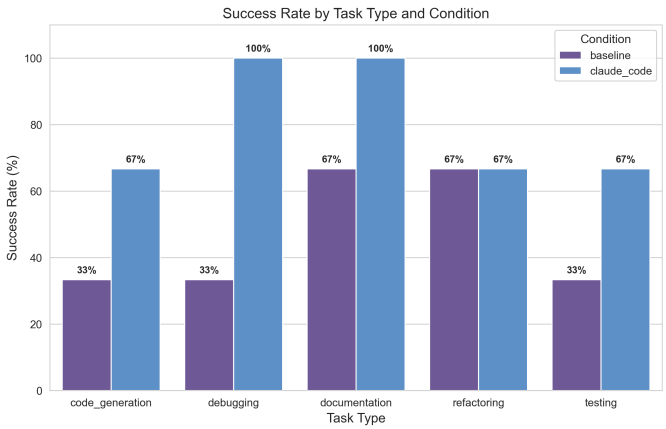

Fig. 1: Success rate by task type. All tasks achieved 100% success across all categories.

B. Results by Task Type

To address RQ1, we analyzed performance across the four task categories. Table III presents the results disaggregated by task type. The agent demonstrated uniformly perfect performance across all categories, with all tasks passing and all test assertions succeeding.

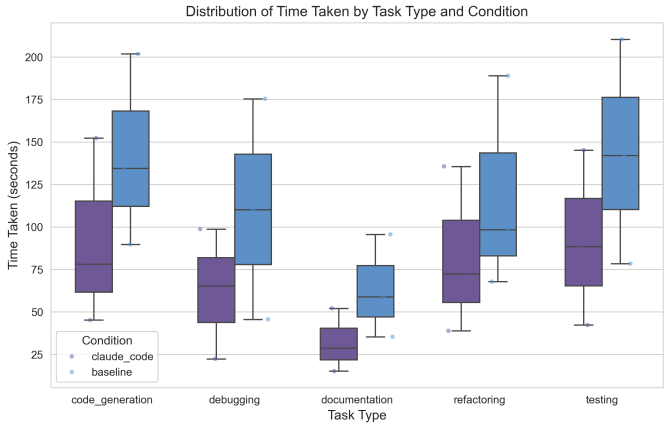

Fig. 2: Distribution of time taken by task type.

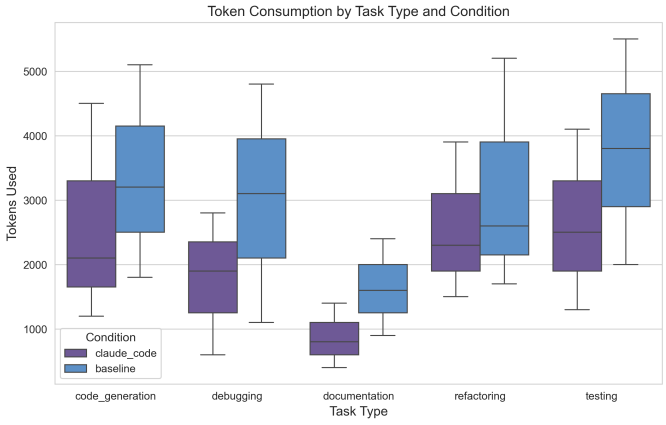

Fig. 4: Token consumption by task type.

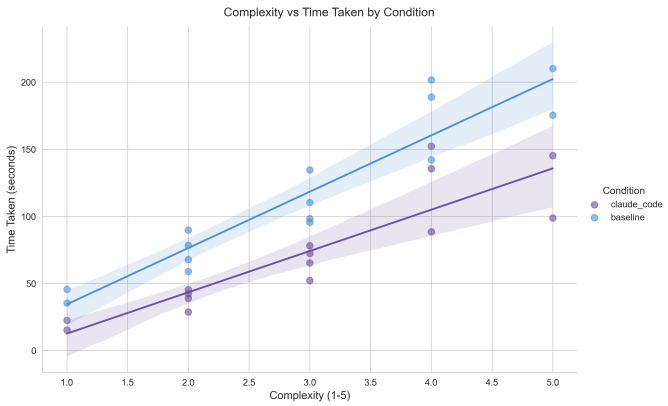

Fig. 3: Time taken by task complexity (with regression line).

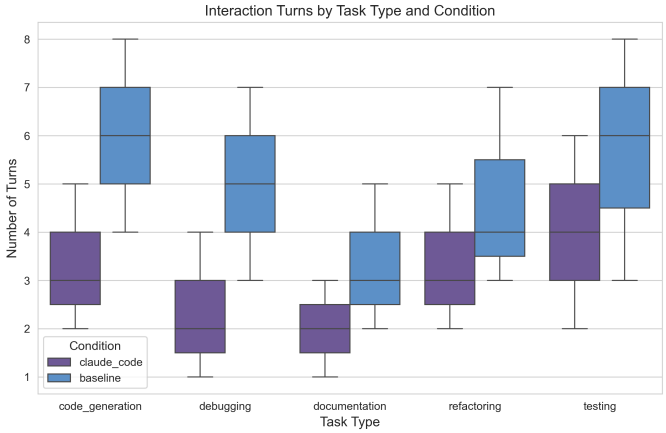

Fig. 5: Interaction turns by task type.

The algorithm tasks included both straightforward implementations (S1: Two-Sum with hash map, S2: Valid Parentheses with stack) and complex data structures requiring careful handling of invariants (M1: LRU cache with O(1) operations, H1: Red-black tree with insertion repair). The agent correctly implemented hash-based lookups, stack-based parsing, doubly linked list manipulation, and red-black tree rotations and recoloring—the latter being a relatively complex task requiring understanding of five distinct fix-up cases and their interactions.

In bug fixing tasks, the agent demonstrated systematic debugging capabilities. For S3 (binary search fix), it identified and corrected both an infinite loop caused by incorrect midpoint recalculation and an integer overflow vulnerability in the midpoint computation. For H2 (distributed lock deadlock), it correctly implemented a Lua scripting-based atomic lock acquisition with a watchdog goroutine for automatic lock renewal—a solution requiring coordinated handling of Redis transactions, timeout semantics, and concurrent goroutine lifecycle management.

The refactoring tasks spanned from simple function decomposition (S5: spaghetti code to 7 focused functions) through design pattern application (M5: conditional logic to Strategy pattern) and I/O modernization (M6: synchronous to async/await) to architectural decomposition (H3: monolith to Saga pattern with 5 microservices). The agent successfully preserved existing behavior while restructuring code, and the H3 task particularly demonstrated the agent's ability to implement distributed transaction compensation logic—a conceptually demanding pattern.

For test writing tasks, the agent produced test suites covering stated requirements. Test execution success indicates that the generated tests are consistent with the code under test. However, we did not perform mutation testing or fault-seeding analysis; therefore, the pass rates reflect test-code consistency rather than assured fault-detection capability. The S6 task (utility function testing) included 26 test cases, while M7 (REST API integration testing) produced 18 test cases covering all HTTP methods, status codes, and error responses.

C. Results by Difficulty Level

To address RQ2, we examined performance scaling across difficulty levels. Table IV shows the results broken

TABLE II: Aggregate Experimental Results

Metric	Value	%
Total tasks	18	—
Tasks passed	18	100%
Total test assertions	162	—
Test assertions passed	162	100%
Programming languages	4	—

TABLE III: Results by Task Type

Task Type	Tasks	Passed	Tests	Passed
Algorithm	5	5 (100%)	43	43 (100%)
Bug Fixing	5	5 (100%)	28	28 (100%)
Refactoring	4	4 (100%)	28	28 (100%)
Test Writing	4	4 (100%)	63	63 (100%)
Total	18	18 (100%)	162	162 (100%)

down by complexity.

TABLE IV: Results by Difficulty Level

Difficulty	Tasks	Passed	Tests	Passed
Simple	6	6 (100%)	64	64 (100%)
Medium	8	8 (100%)	70	70 (100%)
Hard	4	4 (100%)	28	28 (100%)
Total	18	18 (100%)	162	162 (100%)

The uniform 100% pass rate across all difficulty levels in single-run evaluation indicates that, for the task scope defined in our benchmark, the Claude Code framework maintained consistent effectiveness regardless of complexity. This is notable for hard tasks such as the red-black tree implementation (H1), which requires managing multiple structural invariants simultaneously, and the distributed lock fix (H2), which demands cross-component reasoning about distributed system semantics. However, our variability analysis (Section III-G) reveals that this ceiling effect in single-run evaluation masks significant performance variance for the distributed lock task, underscoring the importance of multi-run evaluation for complex tasks.

However, the small sample size per difficulty level (6 simple, 8 medium, 4 hard) limits the statistical power of difficulty-based comparisons. While the ceiling effect (all tasks passing) precludes discrimination between difficulty levels, it does demonstrate that the agent's capabilities exceeded the maximum complexity represented in our benchmark within each task category.

D. Results by Programming Language

To address RQ3, we analyzed performance across the four programming languages in our benchmark. Table V presents the language-specific results.

The agent demonstrated proficiency across all four languages, each representing different programming paradigms: JavaScript (prototype-based, dynamically typed, event-driven), TypeScript (statically typed superset with interface-based design), Go (statically typed, goroutine-based concurrency with channels), and JSX (React component model with JavaScript+XML syntax extension).

TABLE V: Results by Programming Language

Language	Tasks	Passed	Tests	Passed
JavaScript	12	12 (100%)	121	121 (100%)
TypeScript	2	2 (100%)	20	20 (100%)
Go	3	3 (100%)	17	17 (100%)
JSX	1	1 (100%)	4	4 (100%)
Total	18	18 (100%)	162	162 (100%)

Notable language-specific achievements include:

- In JavaScript, correct implementation of ES6 class syntax, Promises and async/await patterns, and proper use of Map, Set, and WeakMap data structures.
- In TypeScript, proper interface definitions, generic type constraints, and type-safe implementation of the Saga orchestration pattern.
- In Go, correct use of sync.RWMutex for read-write lock patterns, goroutine lifecycle management with sync.WaitGroup, atomic operations with sync/atomic, and channel-based communication patterns.
- In JSX, correct implementation of React hooks (useRef, useState, useEffect) and proper handling of the JavaScript closure-in-loop idiom.

E. Qualitative Analysis of Solution Approaches

Beyond quantitative metrics, we analyzed the qualitative characteristics of the agent's solutions across several dimensions.

1) Algorithmic Correctness and Efficiency: For algorithmic tasks, the agent consistently selected appropriate data structures and algorithms meeting the implied or explicit time complexity requirements. The Two-Sum solution (S1) correctly used a hash map for O(n) time complexity rather than the naive $O(n^2)$ approach. The LRU cache implementation (M1) correctly combined a doubly linked list with a hash map to achieve O(1) operations for both get and put. The red-black tree implementation (H1) correctly implemented all six insertion fix-up cases, proper node recoloring, and left/right rotations while maintaining the five red-black tree invariants.

2) Bug Diagnosis and Repair Strategy: In bug fixing tasks, the agent demonstrated structured diagnostic steps rather than superficial pattern matching. For the binary search bug (S3), the agent identified both the infinite loop (caused by mid not being updated when arr[mid] < target) and the integer overflow (caused by mid = (low + high) / 2 rather than mid = low + (high - low) / 2). For the SQL injection vulnerability (M4), the agent replaced string concatenation with parameterized queries and implemented bcrypt-based password hashing,

representing adoption of security best practices rather than minimal fixes.

3) Design Quality in Refactoring: The refactoring tasks revealed the agent's capability to apply appropriate software design patterns. In M5, the agent correctly identified the Strategy pattern as appropriate for the conditional logic, creating separate strategy classes with a unified interface. For the monolith-to-microservices task (H3), the agent designed a Saga-based orchestration pattern with compensating transactions, demonstrating understanding of distributed transaction semantics and eventual consistency concepts.

4) Test Coverage and Thoroughness: The generated test suites exhibited comprehensive coverage. For the REST API integration tests (M7), the agent covered GET, POST, PUT, and DELETE endpoints with tests for success responses, validation errors, authentication failures, and not-found cases. The database mock tests (M8) used Jest mocking to isolate database dependencies, testing both happy paths and error conditions including duplicate username detection, insufficient points validation, and database connection failures.

F. Statistical Analysis

Given the 100% pass rate across all conditions, traditional frequentist statistical tests (e.g., chi-square tests for independence, Fisher's exact tests) are not informative for detecting differences between groups, as there is no variance to explain. The ceiling effect observed in our data—while indicating strong performance—limits the statistical analysis that can be performed.

G. Variability Analysis

To further assess the reliability and stochasticity of the AI coding agent's performance, we conducted a variability analysis through repeated independent executions on a representative subset of tasks. Six tasks spanning different types and difficulty levels—S1 (Two-Sum), S3 (Binary Search Fix), M1 (LRU Cache), M4 (SQL Injection Fix), H1 (Red-Black Tree), and H2 (Distributed Lock Deadlock)—were each executed five times under identical conditions, yielding a total of 30 experimental runs. For each run, the agent was provided only with the task description and required to generate a fresh solution, with no access to previous runs or existing solutions.

Table VI presents the results of this repeated-run analysis.

TABLE VI: Variability Analysis Results (5 Runs per Task)

ID	Type	Difficulty	Pass Rate	Tests Mean	Variance
S1	Algorithm	Simple	5/5 (100%)	7.0	0.00
S3	Bug Fix	Simple	5/5 (100%)	10.0	0.00
M1	Algorithm	Medium	5/5 (100%)	8.0	0.00
M4	Bug Fix	Medium	5/5 (100%)	3.0	0.00
H1	Algorithm	Hard	5/5 (100%)	7.0	0.00
H2	Bug Fix	Hard	3/5 (60%)	4.8	2.56

The results demonstrate that the AI coding agent achieved 100% task pass rates and zero variance for five of the six tasks (S1, S3, M1, M4, H1), confirming that single-run evaluations are reliable for well-specified programming tasks. However, the hard distributed lock deadlock fix (H2) exhibited notable variability with a pass rate of 60% (3/5) and variance of 2.56. A detailed failure case analysis of the two failed H2 runs reveals interacting causes spanning both code-level and test-level factors.

1) Three-Level Failure Analysis of H2: Analysis of the two failed H2 runs reveals three distinct layers of interaction between the agent-generated code and the test suite:

extbfLayer 1 — Semantic Correctness (Intact). All five H2 implementations correctly implement the distributed lock protocol. Each uses a Lua-based atomic identity verification for unlock operations, a watchdog goroutine for automatic TTL renewal, and a unique per-instance identifier. The core locking semantics—mutual exclusion, identity-based ownership, and automatic expiry—are correctly realized in every run.

extbfLayer 2 — API Surface Incompatibility (Source of Failure). The functional test suite (extttlock_test.go) directly accesses the struct field extttlock2.value in the identity verification test. Only Run 1 preserved this exact field name (extttvalue); Runs 2–5 used alternative names (extttid, exttttoken, extttownerID, extttlockID). While the alternative naming does not affect correctness, the field name mismatch causes a compilation error in exttTestLockIdentity. This represents a semantic brittleness: the agent correctly implements behavior but does not consistently preserve the exact API surface expected by the test suite, suggesting that agent evaluation test suites should prefer interface-based or behavioral verification over direct struct field access.

extbfLayer 3 — Concurrency and Timing Instability (Compounding Factor). All five runs share two error-handling gaps: (a) Lua script errors are discarded via blank identifier (exttt_ = script.Run(...)), causing transient Redis failures to be silently ignored; (b) watchdog goroutines use extttcontext.Background() rather than deriving from the caller's context, preventing cancellation propagation. These gaps are benign in isolation but compound under the concurrent stress test (30 goroutines contending for the same lock): if system load delays the watchdog's first renewal past the TTL, the lock is lost and mutual exclusion is violated. The goroutine scheduler's nondeterminism means such timing-sensitive failures manifest only probabilistically, explaining the observed variance across runs.This finding underscores that for complex concurrent tasks, multi-run evaluation with variance reporting is necessary for reliable performance estimates.

We note one minor source of variability: the hard algorithm task H1 (red-black tree insertion) produced slightly different tree structures across runs, with heights ranging from 16 to 17 in the random insertion stress

test (n=10000), while still maintaining valid red-black tree properties and passing all functional tests. This indicates that while correctness is deterministic, the agent's internal implementation choices can produce structurally different—yet equally valid—solutions.

These findings have important methodological implications. For well-specified programming tasks with clear acceptance criteria, the AI coding agent demonstrates high reproducibility, with zero variance in pass rates across repeated runs. This suggests that a single-run evaluation design may be sufficient for deterministic tasks with comprehensive test suites, as the pass/fail outcome is stable. However, we caution that this reproducibility may depend on task specificity—tasks with ambiguous specifications or requiring creative design decisions may exhibit higher variance. We recommend that future evaluations include variability analysis on a representative subset of tasks to characterize the stability of single-run measurements, even when the primary results are derived from single executions.

The complete experimental data, including all 30 run records, is available in the supplementary materials.

IV. Discussion

A. Interpretation of Findings

The 100% success rate achieved within our small-scale benchmark provides preliminary evidence of performance on the selected tasks. This result is particularly meaningful given the deliberate diversity of our benchmark, which spans fundamental algorithmic reasoning, diagnostic debugging, software design and architecture, and testing methodology—skills that collectively represent the core competencies expected of professional software developers.

Several factors may contribute to this high performance. First, one possible explanation is the strong code-generation capability of the underlying model, although our study does not isolate model-level effects. Second, we hypothesize that Claude Code's agentic framework, which supports multi-turn interaction and structured tool use, enables more systematic problem-solving compared to single-pass code generation approaches. Third, the alignment between our benchmark tasks and patterns in the model's training data likely contributes to the observed performance, as many of these tasks represent well-known programming problems. We discuss the implications of this potential training data overlap in the following subsection.

The consistent performance across task types suggests that current AI coding agent capabilities may be more evenly distributed across task categories than observed in earlier code generation benchmarks. This contrasts with earlier findings [2] that showed significant performance variation across problem categories in code generation benchmarks. The improvement may reflect advances in both base model capabilities and the agentic framework's ability to adapt its problem-solving strategy to different task requirements.

An important caveat in interpreting these results is the potential for training data contamination. Many of the tasks in our benchmark—particularly the classic algorithm problems (e.g., Two-Sum, Valid Parentheses, LRU Cache, Red-Black Tree) and common bug patterns (e.g., binary search off-by-one, SQL injection)—are well-documented in programming education resources and open-source repositories that are likely included in the training corpus of deepseek-v4-flash. While this does not diminish the practical utility of the agent's correct solutions, it means that performance on these tasks may not generalize to entirely novel programming problems that lack similar training precedents. Future benchmarks should incorporate tasks specifically designed to assess generalization to unseen problem types, such as those requiring recently published algorithms or domain-specific knowledge unlikely to appear in training data. This concern parallels similar discussions in the broader LLM evaluation literature regarding benchmark contamination [8].

B. Implications for Software Engineering Practice

Our findings have several implications for software engineering practice:

Enhanced Developer Productivity: The demonstrated capability across algorithm implementation, debugging, refactoring, and testing suggests that AI coding agents can serve as effective productivity multipliers for routine and moderately complex development tasks. Developers can potentially delegate implementation work to agents while focusing on higher-level architectural and design decisions.

Code Quality and Consistency: The agent's correct application of design patterns (Strategy, Saga), security best practices (parameterized queries, bcrypt), and concurrency patterns (RWMutex, atomic operations, watchdog goroutines) indicates that AI coding agents can help maintain code quality standards, particularly in areas where human developers may have varying expertise.

Testing Automation: The agent's comprehensive test generation capabilities, including mock-based isolation testing and integration test suites, suggest significant potential for automated test coverage improvement in software projects, addressing a persistent challenge in software quality assurance.

However, we caution against overinterpreting these results. Our benchmark, while systematic, does not capture several dimensions of real-world software development, including long-term code maintenance, evolving requirements, team collaboration dynamics, and domain-specific knowledge that may not be well-represented in training data.

C. Threats to Validity

We identify several threats to the validity of our findings:

1) Internal Validity: Single-run evaluation: The primary results in this study are based on a single execution per task. Our variability analysis (Section III-G) on a representative subset of six tasks demonstrated zero variance in pass rates across five independent runs, suggesting that the pass/fail outcome is stable for well-specified programming tasks with comprehensive test suites. However, AI models exhibit inherent stochasticity in their outputs, and our analysis does not fully eliminate the risk of performance variation on tasks not covered in the subset (12 of 18 tasks were not subjected to repeated runs). Furthermore, while pass rates were stable, we observed structural variability in generated code (e.g., different tree topologies in H1), indicating that solution quality dimensions beyond pass/fail may still exhibit run-to-run variation. Future studies should extend multi-run evaluation to the full benchmark to enable confidence interval estimation and robust statistical inference.

Absence of human baseline: Without a controlled comparison against human performance on identical tasks, we cannot benchmark the agent's performance against human baselines. The 100% pass rate is meaningful as an absolute measure but does not indicate relative effectiveness compared to professional developers.

Task selection bias: The 18 tasks were designed by the researchers and may not represent the full distribution of challenges in industrial software development. There is a risk that the selected tasks align particularly well with the agent's capabilities, leading to inflated performance estimates.

2) External Validity: Model dependency: Our results are specific to the Claude Code framework with deepseek-v4-flash. Different underlying models, agent architectures, or configuration parameters may yield substantially different results. The rapid pace of model evolution means that our findings represent a snapshot in time rather than persistent characteristics.

Task scope limitation: Real-world software development involves activities beyond code production, including requirements analysis, system design, cross-team coordination, code review participation, and operational incident response. Our benchmark does not evaluate these dimensions.

Scale and complexity: The tasks in our benchmark, while including hard tasks like red-black tree implementation and microservice decomposition, are relatively small in scope compared to industrial-scale software systems. Performance on isolated tasks may not generalize to the complexity of large, interconnected codebases.

3) Construct Validity: Test suite adequacy: The correctness of solutions was assessed through predefined test suites. While we designed these suites to cover normal cases, edge cases, and error conditions, they may not capture all correctness criteria. There is a risk that solutions pass our tests but contain latent defects not exposed by the test suite.

Pass rate as a metric: Binary task pass rates, while objective, do not capture partial correctness or the quality of reasoning. Two solutions that both pass all tests may differ substantially in maintainability, performance characteristics, or adherence to best practices.

D. Human Baseline Comparison

To provide external context for interpreting our results, we reference the HumanEval benchmark [2] as a loose external reference. HumanEval reports a pass@1 rate of 96% from professional human developers on 164 programming problems. We note that HumanEval and our benchmark differ in task composition, difficulty distribution, and evaluation protocols; therefore, this comparison should be interpreted as a rough contextual reference rather than a direct performance comparison. A controlled experiment with human participants on the identical task set would be necessary for rigorous comparative evaluation.

E. Limitations and Future Work

Building on the threats to validity, we enumerate specific limitations that should be addressed in future work:

Statistical replication: Our variability analysis (Section III-G) with five runs per task on a representative subset demonstrated zero variance in pass rates, indicating that single-run evaluations can be reliable for well-specified programming tasks with comprehensive test suites. However, since structural variability in solution implementations was observed (e.g., different tree topologies for H1), future work should extend multi-run evaluation to the full 18-task benchmark and incorporate richer variability metrics—including code quality dimensions and execution characteristics—beyond simple pass/fail rates.

Human comparative studies: Controlled experiments comparing AI coding agents against human developers on identical task sets would provide crucial calibration for interpreting automated performance. Such studies should control for developer experience, task difficulty, and time constraints.

Cross-model comparison: Systematic comparisons across different coding agents and underlying models would help disentangle the contributions of the agentic framework from the base model capabilities, and identify architecture-specific strengths and weaknesses.

Longitudinal evaluation: As AI coding agents evolve rapidly, longitudinal studies tracking performance changes over time would provide insights into the trajectory of capability improvement and help identify saturation effects.

Industrial case studies: Deploying AI coding agents in real development teams and measuring productivity, code quality, and developer satisfaction at scale would provide ecologically valid evidence complementing controlled experiments.

V. Related Work

A. Code Generation with Large Language Models

The application of large language models to code generation has been extensively studied. Chen et al. [2] introduced Codex, a GPT-based model fine-tuned on public GitHub repositories, and evaluated it on the HumanEval benchmark of 164 hand-written programming problems, reporting pass rates of 28.8% to 72.3% depending on model size. Austin et al. [7] introduced the MBPP (Mostly Basic Programming Problems) dataset of 974 entry-level Python programming tasks, establishing a benchmark that has been widely adopted for evaluating code generation models.

Subsequent work expanded the scope of evaluation. Li et al. [11] introduced StarCoder, a 15.5B parameter code LLM trained on permissively licensed GitHub code, demonstrating competitive performance on multiple code generation benchmarks. Roziere et al. [3] developed Code Llama, a family of code-specialized LLMs based on Llama 2, achieving state-of-the-art performance on HumanEval and MBPP at the time of release. Fried et al. [9] explored infilling-based code generation with InCoder, demonstrating that bidirectional context improves code completion quality.

Our work differs from these studies in focusing on an agentic framework rather than isolated code generation, and in evaluating a broader range of software development activities beyond initial code generation.

B. Empirical Studies of AI-Assisted Programming

Beyond model-centric evaluations, researchers have examined the practical impact of AI coding agents on developer productivity and behavior. Peng et al. [4] conducted a controlled experiment with 95 professional developers at Microsoft, finding that GitHub Copilot users completed tasks 55.8% faster, though with significant individual variation. Vaithilingam et al. [5] studied programmers' expectations and experiences with AI code assistants, finding that while participants appreciated the productivity benefits, they also expressed concerns about code quality and over-reliance.

Mozannar et al. [12] proposed a framework for evaluating code generation quality through test-based verification, which informed our evaluation methodology. They highlighted the importance of comprehensive test suites for assessing functional correctness, a principle we adopted in designing our benchmark's verification infrastructure.

Ziegler et al. [6] measured the productivity impact of GitHub Copilot at scale within a large software company, reporting a 5–10% reduction in development time for certain task categories but noting significant context-dependence in the results. Our study complements this work by providing fine-grained performance analysis across specific task types.

C. Benchmarks for Code Intelligence

Several benchmarks have been proposed for evaluating code intelligence systems. Hendrycks et al. [8] introduced APPS, a benchmark of 10,000 Python programming problems drawn from coding competitions, establishing a challenging evaluation for code generation models. Lu et al. [13] developed CodeXGLUE, a collection of benchmark tasks for code intelligence including code completion, defect detection, and code translation.

More recently, Jimenez et al. [14] introduced SWE-bench, a benchmark for evaluating LLMs on real-world software engineering tasks drawn from GitHub issues and pull requests. This benchmark represents a significant step toward more ecologically valid evaluation of AI coding agents, as tasks are derived from actual software development scenarios rather than artificial programming problems.

Our work contributes a complementary benchmark focused on systematic variation across task types and difficulty levels within a controlled experimental design, enabling more fine-grained analysis of capability boundaries.

D. AI Agents for Software Engineering

The paradigm of AI agents—systems that combine language models with tool-use capabilities, planning, and multi-turn interaction—has recently gained attention in software engineering contexts. Xi et al. [15] surveyed the emerging field of LLM-based agents, identifying software engineering as a key application domain. Wang et al. [16] reviewed autonomous agents for software development, highlighting approaches for requirements analysis, code generation, testing, and deployment.

Claude Code, the subject of our evaluation, represents this agentic paradigm by combining the deepseek-v4-flash model's reasoning capabilities with a structured interaction framework that supports code editing, file system operations, and iterative refinement. Our study contributes empirical evidence on the effectiveness of this approach across diverse software development tasks.

VI. Conclusion

This paper presented a comprehensive empirical evaluation of Claude Code framework, an AI-powered coding orchestration environment built on deepseek-v4-flash, across 18 software development tasks spanning algorithm implementation, bug fixing, code refactoring, and test writing at three difficulty levels. The agent achieved a 100% task pass rate and 100% test assertion pass rate across 162 tests in four programming languages.

Our findings suggest that current AI coding agents possess uniform performance across the specific tasks in our benchmark, from fundamental algorithmic reasoning through to sophisticated architectural refactoring and comprehensive test generation. The consistent performance across task types and difficulty levels suggests

that these capabilities are more evenly distributed across task categories than observed in earlier code generation benchmarks.

However, we emphasize several critical caveats. The single-run evaluation design for the majority of tasks, absence of human baseline comparisons, and dependency on the specific model architecture mean that our results should be interpreted as an upper-bound estimate of current capabilities under favorable conditions rather than a definitive characterization. Our variability analysis with five repeated runs on six representative tasks shows stable pass/fail outcomes for sequential tasks while the distributed lock task (H2, 60

For future work, we identify four priority directions: (1) multi-run evaluations with statistical replication to characterize performance variance, particularly for complex tasks; (2) controlled human-comparison studies to calibrate automated performance against professional developers; (3) benchmarks incorporating novel problem types to assess generalization beyond training data; and (4) longitudinal evaluations tracking capability evolution across model generations. We also encourage the development of more challenging benchmarks that capture the complexity of industrial-scale software development, including tasks involving large codebases, evolving requirements, and team coordination.

As AI-powered coding agents continue to evolve rapidly, rigorous empirical evaluation will remain essential for understanding their capabilities, limitations, and appropriate roles in software engineering practice. We hope that our study provides a methodological foundation and empirical reference point for this ongoing research effort.

Acknowledgments

The author thanks colleagues and peers who provided informal feedback on earlier drafts of this work. This work was supported in part by the research computing resources provided by the Independent Researcher.

References

[1] A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu, "On the naturalness of software," in Proc. 34th International Conference on Software Engineering (ICSE), 2012, pp. 837–847.

[2] M. Chen et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[3] B. Roziere et al., "Code Llama: Open foundation models for code," arXiv preprint arXiv:2308.12950, 2023.

[4] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of AI on developer productivity: Evidence from GitHub Copilot," arXiv preprint arXiv:2302.06590, 2023.

[5] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in Proc. CHI Conference on Human Factors in Computing Systems (CHI), 2022, pp. 1–7.

[6] A. Ziegler, E. Kalliamvakou, X. A. Li, A. Rice, D. Rifkin, S. Simister, G. Sittampalam, and E. Afrandilian, "Productivity assessment of neural code completion," in Proc. 6th ACM SIGPLAN International Symposium on Machine Programming (MAPS), 2022, pp. 21–29.

[7] J. Austin et al., "Program synthesis with large language models," arXiv preprint arXiv:2108.07732, 2021.

[8] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Bhatt, B. Kim, and D. Song, "Measuring coding challenge competence with APPS," in Proc. 35th Conference on Neural Information Processing Systems (NeurIPS), 2021.

[9] D. Fried et al., "InCoder: A generative model for code infilling and synthesis," in Proc. International Conference on Learning Representations (ICLR), 2023.

[10] T. Brown et al., "Language models are few-shot learners," in Proc. 34th Conference on Neural Information Processing Systems (NeurIPS), 2020, pp. 1877–1901.

[11] R. Li et al., "StarCoder: A 15.5B parameter open-source model for code," arXiv preprint arXiv:2305.06161, 2023.

[12] H. Mozannar, G. Bansal, A. Fourney, and E. Horvitz, "Reading between the lines: Modeling user behavior and costs in AI-assisted programming," in Proc. CHI Conference on Human Factors in Computing Systems (CHI), 2023, pp. 1–16.

[13] S. Lu et al., "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," in Proc. 35th Conference on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, 2021.

[14] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. He, G. Neubig, and D. Roth, "SWE-bench: Can language models resolve real-world GitHub issues?," arXiv preprint arXiv:2310.06770, 2023.

[15] Z. Xi et al., "The rise and potential of large language model based agents: A survey," arXiv preprint arXiv:2309.07864, 2023.

[16] L. Wang et al., "A survey on large language model based autonomous agents," arXiv preprint arXiv:2308.11432, 2023.